

Laporan Praktikum Internet of Things (IoT)

Simulasi Pemantauan Suhu dan Kelembaban Berbasis IoT Menggunakan ESP32 dan NodeRed

Cagita Dian A'yunin,
Fakultas Vokasi, Universitas Brawijaya
cagitadian@student.ub.ac.id

Abstrak

Praktikum ini merupakan praktikum perancangan dan simulasi sistem monitoring suhu dan kelembapan berbasis Internet of Things (IoT) menggunakan mikrokontroler ESP32 dan sensor DHT22, yang dikembangkan secara virtual melalui platform Wokwi dan Visual Studio Code. Sistem menggunakan broker `broker.emqx.io`. Data yang dihasilkan oleh sensor dikirim melalui topik dengan terdapat dua topik yakni humidity dan temperature lalu divisualisasikan menggunakan Node-RED. Node-RED berperan dalam mengelola alur data, mulai dari penerimaan data sensor, pemrosesan, penyimpanan menggunakan InfluxDB sebagai database time-series, dan penyajian data secara visual dalam bentuk grafik dan indikator (gauge) baik dari kedua topik. Selain itu, sistem juga mampu menerima data masuk dari broker, misalnya untuk menyalakan atau mematikan LED sebagai bentuk interaksi dua arah (publish-subscribe). Pada praktikum ini dilakukan tanpa perangkat fisik sehingga dapat lebih memudahkan dalam melakukan percobaan sebelum turun ke perangkat keras asli. Hasil simulasi menunjukkan bahwa sistem berjalan dengan stabil, responsif, dan dapat menangani pengiriman maupun penerimaan data secara real-time.

Kata Kunci—Internet of Things, Wokwi, NodeRed, DHT22

Abstract

This practicum is a design and simulation practicum of an Internet of Things (IoT)-based temperature and humidity monitoring system using an ESP32 microcontroller and a DHT22 sensor, developed virtually through the Wokwi platform and Visual Studio Code. The system uses broker `broker.emqx.io`. The data generated by the sensor is sent through topics with two topics namely humidity and temperature and then visualized using Node-RED. Node-RED plays a role in managing data flow, starting from receiving sensor data, processing, storing using InfluxDB as a time-series database, and presenting data visually in the form of graphs and indicators (gauges) from both topics. In addition, the system is also able to receive incoming data from brokers, for example to turn on or off the LED as a form of two-way interaction (publish-subscribe). This practicum is done without physical devices so that it can make it easier to do experiments before going down to real hardware. The simulation results show that the system runs stably, responsively, and can handle sending and receiving data in real-time.

Keywords— Internet of Things, Wokwi, NodeRed, DHT22

1. Introduction (Pendahuluan)

Pemantauan suhu dan kelembapan semakin dibutuhkan dalam kehidupan sehari-hari. Dengan berkembangnya teknologi Internet of Things (IoT), proses pemantauan dapat dilakukan secara otomatis, dan real-time. Pada praktikum ini, sistem monitoring suhu dan kelembapan dirancang menggunakan ESP32 sebagai mikrokontroler utama dan sensor DHT22 sebagai pengambil data lingkungan. Komunikasi data dilakukan melalui protokol MQTT dan divisualisasikan menggunakan Node-RED. Praktikum ini bertujuan untuk memahami alur kerja, mulai dari pembacaan data sensor hingga penyajiannya dalam bentuk visual.

1.1 Latar Belakang

Di era digital saat ini, pengumpulan data lingkungan secara otomatis sangat dibutuhkan untuk mendukung pengambilan keputusan yang cepat dan akurat. Oleh karena itu, dibutuhkan sistem pemantauan berbasis IoT yang mampu mengirim dan menampilkan data secara real-time. Praktikum ini dilaksanakan secara virtual menggunakan Wokwi, MQTT sebagai protokol komunikasi ringan, Node-RED sebagai alat visualisasi data, serta InfluxDB dalam menyimpan data historis. Sistem yang bisa berjalan secara otomatis akan sangat dibutuhkan di berbagai industri sehingga praktikum ini dapat menggambarkan terkait bagaimana cara membangun sistem kerja IoT.

1.2 Tujuan Eksperimen

Eksperimen ini bertujuan untuk:

- Menerapkan konsep Internet of Things (IoT) dalam sistem monitoring suhu dan kelembapan.
- Memberikan pemahaman terkait integrasi sistem dengan Node-RED untuk mengelola alur data dan memvisualisasikan data sensor dalam bentuk grafik dan indikator.

2. Methodology (Metodologi)

2.1 Tools & Materials (Alat dan Bahan)

Untuk menjalankan eksperimen ini, beberapa alat dan bahan yang digunakan adalah:

- Platform simulasi Wokwi: Digunakan untuk melakukan simulasi sistem secara digital.
- Visual Studio Code dengan ekstensi PlatformIO dengan library
- Kode program berbasis C/C++: Digunakan untuk mengontrol urutan nyala-mati LED virtual dalam simulasi.
- Node Red : pengelola alur dan visualisasi data
- Node Red nodes : text, function, inject, gauge, chart, debug, node-red-dashboard (interface), node-red-contrib-influxdb (koneksi ke InfluxDB), dan lain lain
- Mikrokontroler ESP32 (virtual pada Wokwi dan PlatformIO): Sebagai unit pemrosesan utama dan sebagai pengendali yang dilengkapi dengan Wi-Fi dan Bluetooth.
- ESP32 DevKit (Virtual) : Mikrokontroler untuk membaca dan mengirimkan data sensor
- Sensor DHT22 (Virtual) : Sensor yang digunakan untuk membaca suhu dan kelembapan.
- LED RGB (Virtual) : Indikator dan dikontrol melalui dashboard Blynk.
- Resistor (Virtual)
- Library : DHT sensor library for ESPx, PubSubClient

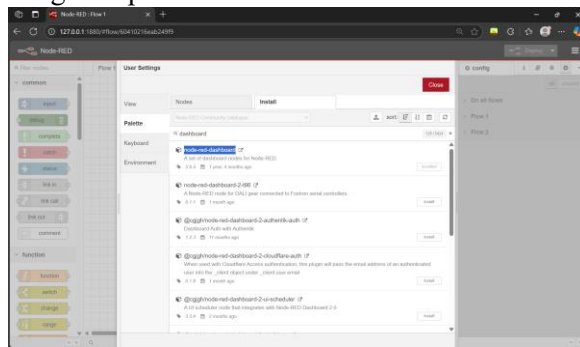
2.2 Implementation Steps (Langkah Implementasi)

- Buat project baru di Visual Studio Code dengan project baru melewati PlatformIO.
- Buat project di wokwi simulator.
- Tambahkan perangkat lalu sambungkan dan tambahkan library yang dibutuhkan.
- Tambahkan kode program lalu uji.
- Buka Command Prompt lalu install `npm install -g -unsafe-perm node-red`
- Run dengan node-red lalu masuk ke dalam Alamat ip yang muncul

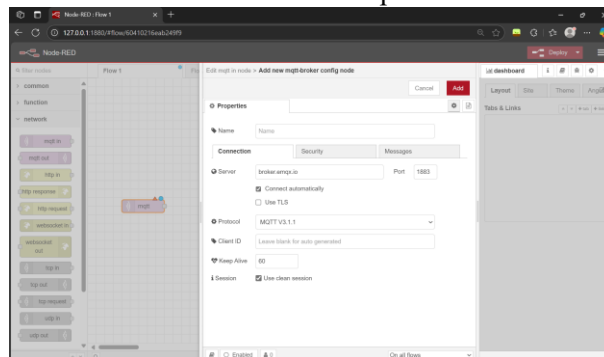
```
12 Jun 15:24:01 - [warn] Encrypted credentials not found
12 Jun 15:24:01 - [info] Server now running at http://127.0.0.1:1880/
12 Jun 15:24:01 - [info] Starting flows
12 Jun 15:24:01 - [info] Started flows
```

- Lalu cari menu setting di tanda burger di pojok kanan atas

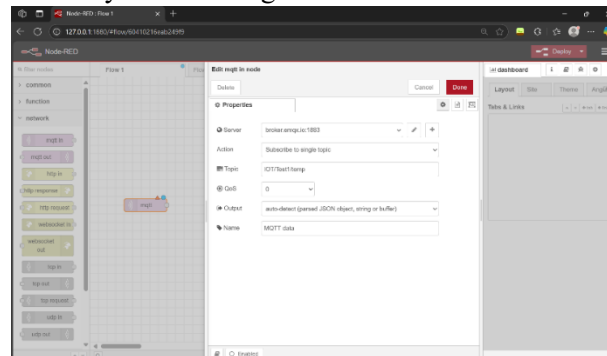
- Install dashboard pallete node-red-dashboard secara langsung ataupun melewati cmd dengan “npm install node-red-dashboard” lalu close



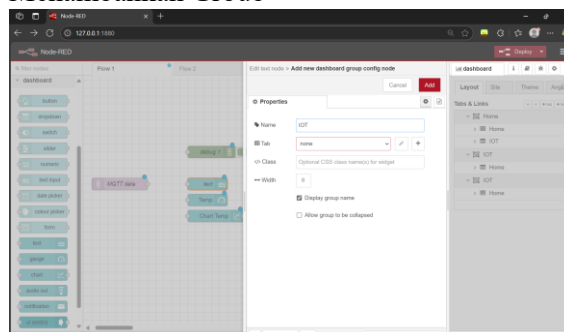
- Menampilkan data dari sensor
 - Pada filter nodes pilih Network lalu pilih mqtt in
 - Klik dua kali lalu lengkapi setiap bagian yang kosong, dari tambahkan server terlebih dahulu. Berikut adalah proses menambahkan server lalu tekan add



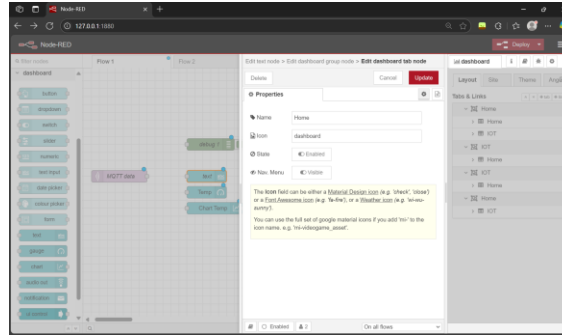
- Topic dan server di samakan dengan project DHT-MQTT : <https://wokwi.com/projects/433568890937503745>
- Isi sisanya sesuai dengan di bawah ini lalu tekan done



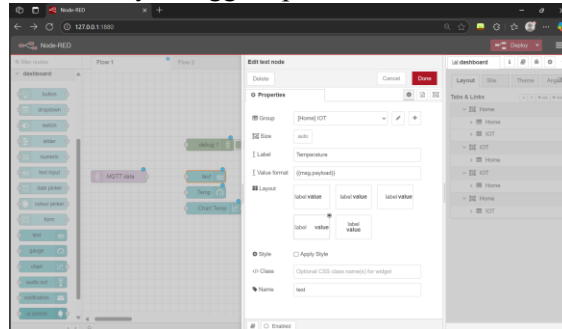
- Pada nodes pilih common, lalu pilih debug
 - Pada nodes pilih dashboard, lalu pilih text
- Menambahkan Group



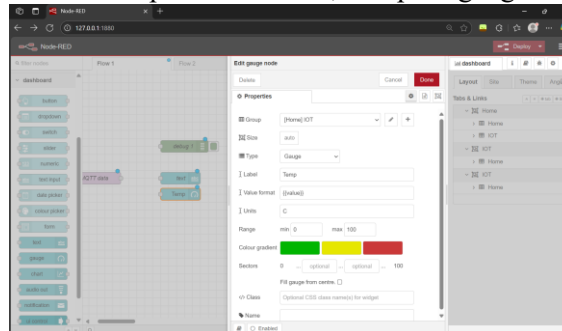
Menambahkan Tab



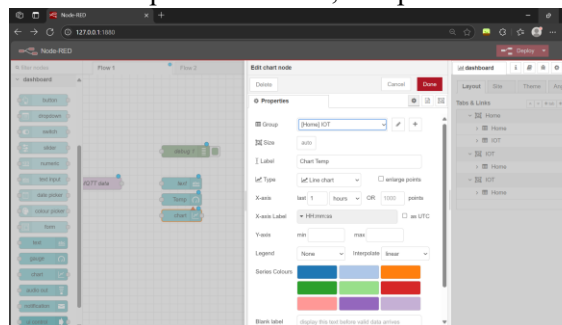
Isi semuanya hingga seperti ini



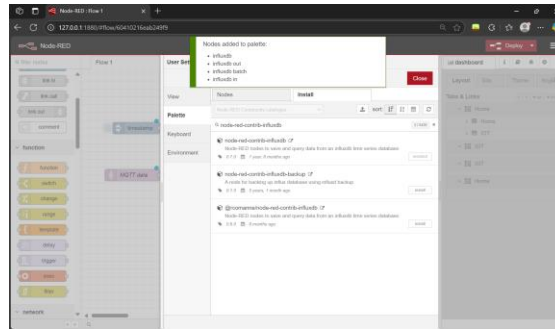
- Pada nodes pilih dashboard, lalu pilih gauge



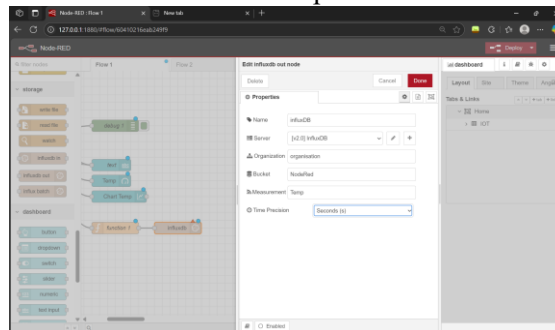
- Pada nodes pilih dashboard, lalu pilih chart



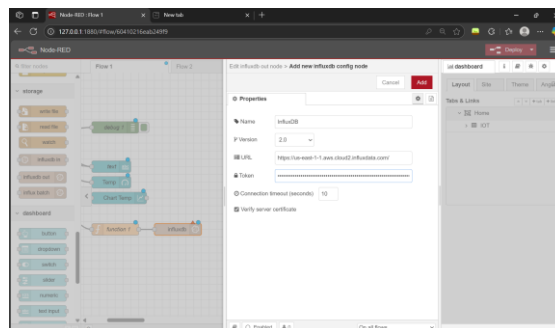
- Pada nodes pilih function, lalu pilih function
- Pada nodes pilih common, lalu pilih debug
- Pada nodes pilih common, lalu pilih inject
- Klik tombol menu kanan atas (≡) > Pilih **"Manage palette"** > Buka tab **Install** > Cari node-red-contrib-influxdb > Klik **Install**. Setelah itu akan muncul filter nodes storage lalu pilih influxdb out



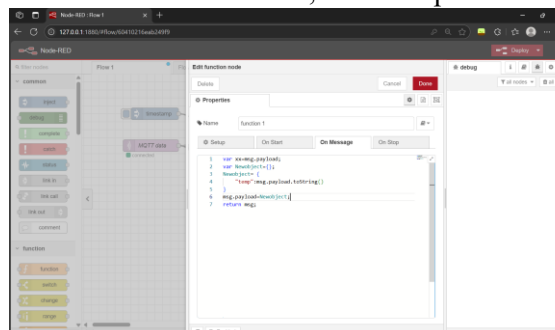
- Menggunakan database : influxdb
 - Buka situ resminya lalu Sign In dan pilih Log In to InfluxDB Cloud 2.0 lalu ikuti proses pendaftarannya
 - Pilih manage databases & Security > Access Token Manager > Go To Tokens > Generate API Token > All Access API Token > Copy Token
 - Masuk ke Bucket > Create Bucket > Kasih nama NodeRed
 - Influxdb out masukkan seperti ini



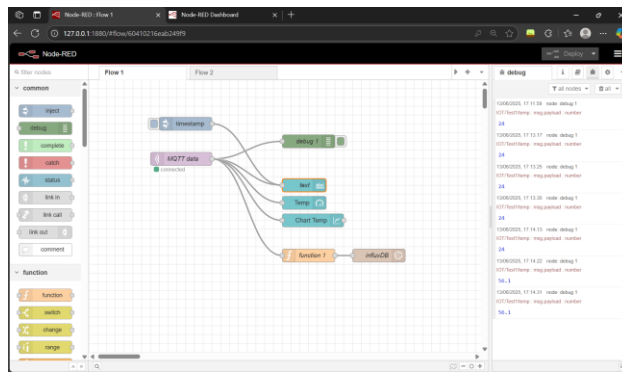
- Di bawah ini Server



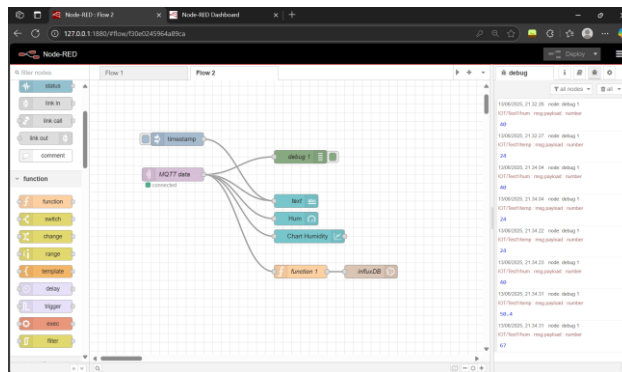
- Masuk ke dalam function, lalu isi seperti ini



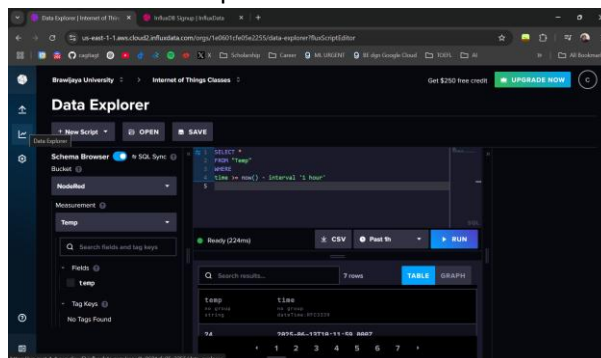
- Lalu hubungkan setiap nodes seperti ini



- Buat flow yang berbeda lalu lakukan hal sama untuk Humidity dan penting untuk mengubah di MQTT data pada bagian Topic untuk mengganti menjadi IOT/Test1/hum



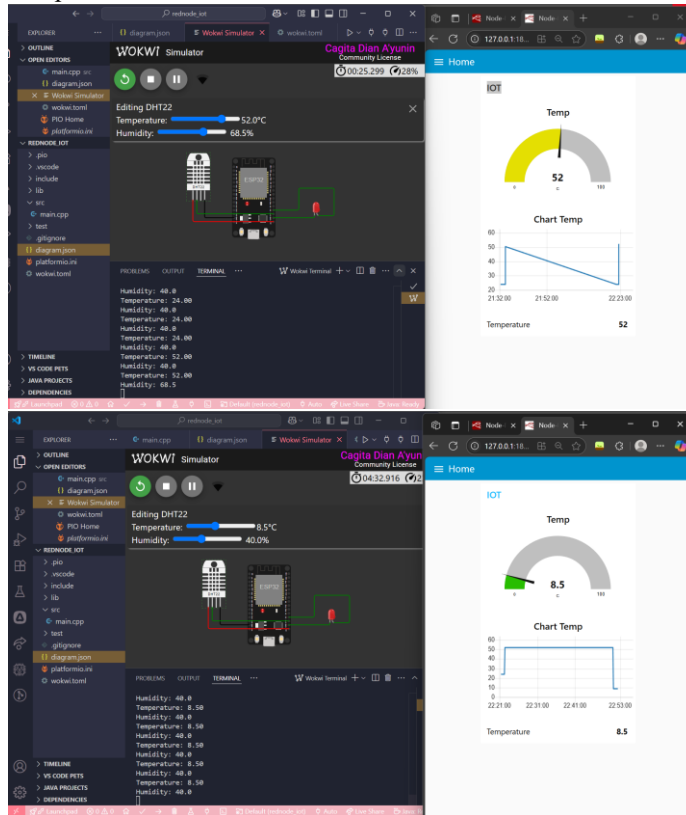
- Deploy
- Masuk ke <http://127.0.0.1:1880/ui>
- Lalu coba running di project DHT-MQTT : <https://wokwi.com/projects/433568890937503745>
- Masuk ke web Influxdb lalu ke bagian data explorer dan lakukan seperti di bawah ini. Jangan lupa menyalakan SQL Sync lalu coba run, untuk menunjukkan data telah masuk dan tersimpan



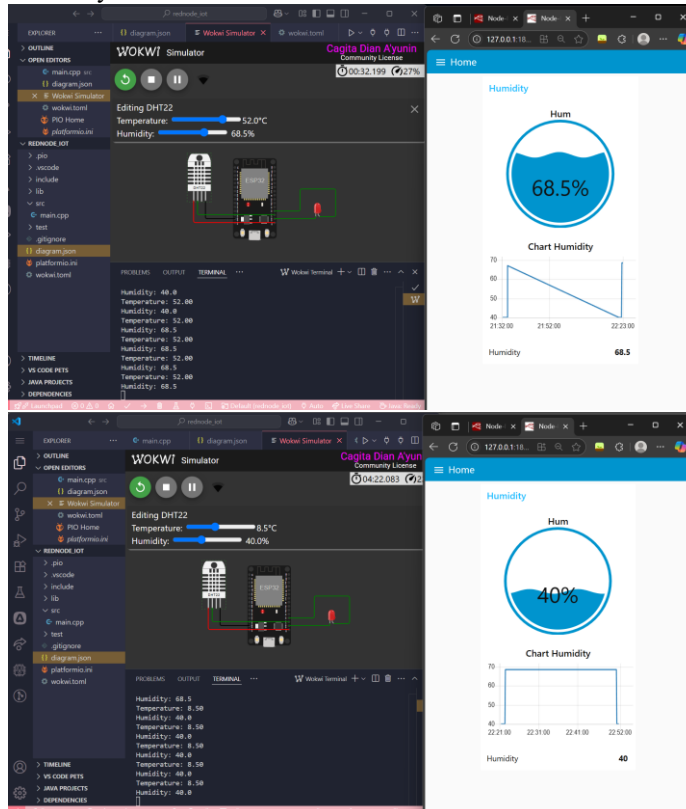
3. Result dan Discussion (Hasil dan Pembahasan)

3.1 Experimental Result (Hasil Eksperimen)

Berikut adalah screenshot hasil simulasi pada Visual Studio Code :
Temperature



Humidity



Hasil praktikum monitoring suhu dan kelembaban menunjukkan bahwa dashboard dapat mengirimkan data secara realtime dari folder project baik dari wokwi maupun Visual Studio Code ke dashboard tampilan yang sudah diatur diagram alurnya dan konfigurasi setiap bagian. Dashboard terbagi menjadi dua halaman yakni halaman untuk IOT/Temperature dan halaman Humidity, keduanya memiliki alur pengejerjaan yang sama tetapi yang membedakan adalah topic yang dipilih berbeda dan untuk elemen lainnya disesuaikan sesuai kebutuhan masing-masing.

- Temperature : IOT/Test1/temp
- Humidity : IOT/Test1/hum

4. Appendix (Lampiran)

4.1 Kode Program (Main.cpp)

```
#include <Arduino.h>
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHTesp.h>

const int LED_RED = 2;
const int DHT_PIN = 15;
DHTesp dht;

// Update these with values suitable for your network.

const char* ssid = "Wokwi-GUEST";
const char* password = "";
const char* mqtt_server = "broker.emqx.io";

WiFiClient espClient;
PubSubClient client(espClient);
unsigned long lastMsg = 0;
float temp = 0;
float hum = 0;

void setup_wifi() { //perintah koneksi wifi
  delay(10);
  // We start by connecting to a WiFi network
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);

  WiFi.mode(WIFI_STA); //setting wifi chip sebagai station/client
  WiFi.begin(ssid, password); //koneksi ke jaringan wifi

  while (WiFi.status() != WL_CONNECTED) { //perintah tunggu esp32 sampai terkoneksi ke wifi
    delay(500);
    Serial.print(".");
  }

  randomSeed(micros());

  Serial.println("");
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
}
```



```

void callback(char* topic, byte* payload, unsigned int length) { //perintah untuk menampilkan
data ketika esp32 di setting sebagai subscriber
  Serial.print("Message arrived [");
  Serial.print(topic);
  Serial.print("] ");
  for (int i = 0; i < length; i++) { //mengecek jumlah data yang ada di topik mqtt
    Serial.print((char)payload[i]);
  }
  Serial.println();

  // Switch on the LED if an 1 was received as first character
  if ((char)payload[0] == '1') {
    digitalWrite(LED_RED, HIGH); // Turn the LED on
  } else {
    digitalWrite(LED_RED, LOW); // Turn the LED off
  }
}

void reconnect() { //perintah koneksi esp32 ke mqtt broker baik itu sebagai publusher atau
subscriber
  // Loop until we're reconnected
  while (!client.connected()) {
    Serial.print("Attempting MQTT connection...");
    // perintah membuat client id agar mqtt broker mengenali board yang kita gunakan
    String clientId = "ESP32Client-";
    clientId += String(random(0xffff), HEX);
    // Attempt to connect
    if (client.connect(clientId.c_str())) {
      Serial.println("Connected");
      // Once connected, publish an announcement...
      client.publish("IOT/Test1/mqtt", "Test IOT"); //perintah publish data ke alamat topik yang di
setting
      // ... and resubscribe
      client.subscribe("IOT/Test1/mqtt"); //perintah subscribe data ke mqtt broker
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      // Wait 5 seconds before retrying
      delay(5000);
    }
  }
}

void setup() {
  pinMode(LED_RED, OUTPUT); // inisialisasi pin 2 / ledbuiltin sebagai output
  Serial.begin(115200);
  setup_wifi(); //memanggil void setup_wifi untuk dieksekusi
  client.setServer(mqtt_server, 1883); //perintah connecting / koneksi awal ke broker
  client.setCallback(callback); //perintah menghubungkan ke mqtt broker untuk subscribe data
  dht.setup(DHT_PIN, DHTesp::DHT22); //inisialiasi komunikasi dengan sensor dht22
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
}

```

```

    }
    client.loop();

    unsigned long now = millis();
    if (now - lastMsg > 2000) { //perintah publish data
        lastMsg = now;
        TempAndHumidity data = dht.getTempAndHumidity();

        String temp = String(data.temperature, 2); //membuat variabel temp untuk di publish ke broker
        mqtt
        client.publish("IOT/Test1/temp", temp.c_str()); //publish data dari varibel temp ke broker mqtt

        String hum = String(data.humidity, 1); //membuat variabel hum untuk di publish ke broker mqtt
        client.publish("IOT/Test1/hum", hum.c_str()); //publish data dari varibel hum ke broker mqtt

        Serial.print("Temperature: ");
        Serial.println(temp);
        Serial.print("Humidity: ");
        Serial.println(hum);
    }
}

```

4.2 Kode Program (diagram.json)

```

{
  "version": 1,
  "author": "Subairi",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 0, "left": 0, "attrs": {} },
    { "type": "wokwi-dht22", "id": "dht1", "top": -9.3, "left": -111, "attrs": {} },
    { "type": "wokwi-led", "id": "led1", "top": 102, "left": 186.2, "attrs": { "color": "red" } }
  ],
  "connections": [
    [ "esp:TX0", "$serialMonitor:RX", "", [] ],
    [ "esp:RX0", "$serialMonitor:TX", "", [] ],
    [ "dht1:GND", "esp:GND.2", "black", [ "v0" ] ],
    [ "dht1:VCC", "esp:3V3", "red", [ "v0" ] ],
    [ "dht1:SDA", "esp:D15", "green", [ "v0" ] ],
    [ "led1:C", "esp:GND.1", "green", [ "v0" ] ],
    [ "esp:D2", "led1:A", "green", [ "h61.9", "v-53.6", "h86.4", "v57.6" ] ]
  ],
  "dependencies": {}
}

```

4.3 Kode Program NodeRed (IOT/Temperature)

```

[
  {
    "id": "60410216eab249f9",
    "type": "tab",
    "label": "Flow 1",
    "disabled": false,
    "info": "",
    "env": []
  },
  {
    "id": "e0add80ee26d3e14",

```

```

    "type": "influxdb out",
    "z": "60410216eab249f9",
    "influxdb": "1a5d5a3e0b051f28",
    "name": "influxDB",
    "measurement": "Temp",
    "precision": "",
    "retentionPolicy": "",
    "database": "database",
    "precisionV18FluxV20": "s",
    "retentionPolicyV18Flux": "",
    "org": "organisation",
    "bucket": "NodeRed",
    "x": 680,
    "y": 400,
    "wires": []
  },
  {
    "id": "70b6d35c63e4f702",
    "type": "ui_text",
    "z": "60410216eab249f9",
    "group": "9a6f421eb4168685",
    "order": 0,
    "width": 0,
    "height": 0,
    "name": "text",
    "label": "Temperature",
    "format": "{{msg.payload}}",
    "layout": "row-spread",
    "className": "",
    "style": false,
    "font": "",
    "fontSize": 16,
    "color": "#000000",
    "x": 510,
    "y": 240,
    "wires": []
  },
  {
    "id": "24f0f3f0fa94d413",
    "type": "ui_gauge",
    "z": "60410216eab249f9",
    "name": "",
    "group": "9a6f421eb4168685",
    "order": 1,
    "width": 0,
    "height": 0,
    "gtype": "gage",
    "title": "Temp",
    "label": "C",
    "format": "{{value}}",
    "min": 0,
    "max": "100",
    "colors": [
      "#00b500",
      "#e6e600",
      "#ca3838"
    ]
  }

```

```

    ],
    "seg1": "",
    "seg2": "",
    "diff": false,
    "className": "",
    "x": 510,
    "y": 280,
    "wires": []
  },
  {
    "id": "bfd300c88382537e",
    "type": "mqtt in",
    "z": "60410216cab249f9",
    "name": "MQTT data",
    "topic": "IOT/Test1/temp",
    "qos": "0",
    "datatype": "auto-detect",
    "broker": "64ec3c34abd42955",
    "nl": false,
    "rap": true,
    "rh": 0,
    "inputs": 0,
    "x": 230,
    "y": 180,
    "wires": [
      [
        "c0cfb805087f38cf",
        "db0404ffb3dfe9b",
        "70b6d35c63e4f702",
        "24f0f3f0fa94d413",
        "58e4410d398e5c07"
      ]
    ]
  },
  {
    "id": "b03f06c1f7afcb5a",
    "type": "inject",
    "z": "60410216cab249f9",
    "name": "",
    "props": [
      {
        "p": "payload"
      },
      {
        "p": "topic",
        "vt": "str"
      }
    ]
  },
  "repeat": "",
  "crontab": "",
  "once": false,
  "onceDelay": 0.1,
  "topic": "",
  "payload": "",
  "payloadType": "date",
  "x": 240,

```

```

        "y": 100,
        "wires": [
            [
                "70b6d35c63e4f702"
            ]
        ]
    },
    {
        "id": "db0404ffb3dfef9b",
        "type": "debug",
        "z": "60410216eab249f9",
        "name": "debug 1",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "payload",
        "targetType": "msg",
        "statusVal": "",
        "statusType": "auto",
        "x": 520,
        "y": 140,
        "wires": []
    },
    {
        "id": "58e4410d398e5c07",
        "type": "ui_chart",
        "z": "60410216eab249f9",
        "name": "",
        "group": "9a6f421eb4168685",
        "order": 2,
        "width": 0,
        "height": 0,
        "label": "Chart Temp",
        "chartType": "line",
        "legend": "false",
        "xformat": "HH:mm:ss",
        "interpolate": "linear",
        "nodata": "",
        "dot": false,
        "ymin": "",
        "ymax": "",
        "removeOlder": 1,
        "removeOlderPoints": "",
        "removeOlderUnit": "3600",
        "cutout": 0,
        "useOneColor": false,
        "useUTC": false,
        "colors": [
            "#1f77b4",
            "#aec7e8",
            "#ff7f0e",
            "#2ca02c",
            "#98df8a",
            "#d62728",
            "#ff9896",

```

```

        "#9467bd",
        "#c5b0d5"
    ],
    "outputs": 1,
    "useDifferentColor": false,
    "className": "",
    "x": 530,
    "y": 320,
    "wires": [
        []
    ]
},
{
    "id": "c0cfb805087f38cf",
    "type": "function",
    "z": "60410216eab249f9",
    "name": "function 1",
    "func": "var xx=msg.payload;\nvar Newobject={};\nNewobject= {\n\n\t\"temp\":msg.payload.toString()\n}\nmsg.payload=Newobject;\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 520,
    "y": 400,
    "wires": [
        [
            "e0add80ee26d3e14"
        ]
    ]
},
{
    "id": "1a5d5a3e0b051f28",
    "type": "influxdb",
    "hostname": "127.0.0.1",
    "port": 8086,
    "protocol": "http",
    "database": "database",
    "name": "InfluxDB",
    "usetls": false,
    "tls": "",
    "influxdbVersion": "2.0",
    "url": "https://us-east-1-1.aws.cloud2.influxdata.com/",
    "timeout": 10,
    "rejectUnauthorized": true
},
{
    "id": "9a6f421eb4168685",
    "type": "ui_group",
    "name": "IOT",
    "tab": "0d05ed42070f895c",
    "order": 1,
    "disp": true,
    "width": 6,

```

```

    "collapse": false,
    "className": ""
  },
  {
    "id": "64ec3c34abd42955",
    "type": "mqtt-broker",
    "name": "",
    "broker": "broker.emqx.io",
    "port": 1883,
    "clientid": "",
    "autoConnect": true,
    "usetls": false,
    "protocolVersion": 4,
    "keepalive": 60,
    "cleansession": true,
    "autoUnsubscribe": true,
    "birthTopic": "",
    "birthQos": "0",
    "birthRetain": "false",
    "birthPayload": "",
    "birthMsg": {},
    "closeTopic": "",
    "closeQos": "0",
    "closeRetain": "false",
    "closePayload": "",
    "closeMsg": {},
    "willTopic": "",
    "willQos": "0",
    "willRetain": "false",
    "willPayload": "",
    "willMsg": {},
    "userProps": "",
    "sessionExpiry": ""
  },
  {
    "id": "0d05ed42070f895c",
    "type": "ui_tab",
    "name": "Home",
    "icon": "dashboard",
    "disabled": false,
    "hidden": false
  }
]

```

4.4 Kode Program NodeRed (Humidity)

```

[
  {
    "id": "56f82a35049eb55c",
    "type": "mqtt in",
    "z": "f30e0245964a89ca",
    "name": "MQTT data",
    "topic": "IOT/Test1/hum",
    "qos": "0",
    "datatype": "auto-detect",
    "broker": "64ec3c34abd42955",
    "nl": false,

```

```

    "rap": true,
    "rh": 0,
    "inputs": 0,
    "x": 190,
    "y": 300,
    "wires": [
      [
        "964c205282dccf4b",
        "b64d818108bcd78d",
        "0d2a3acacbab6b57",
        "55c003b22b61aa78",
        "e7fb3c0c4cf4ee14"
      ]
    ]
  },
  {
    "id": "64ec3c34abd42955",
    "type": "mqtt-broker",
    "name": "",
    "broker": "broker.emqx.io",
    "port": 1883,
    "clientId": "",
    "autoConnect": true,
    "usetls": false,
    "protocolVersion": 4,
    "keepalive": 60,
    "cleansession": true,
    "autoUnsubscribe": true,
    "birthTopic": "",
    "birthQos": "0",
    "birthRetain": "false",
    "birthPayload": "",
    "birthMsg": {},
    "closeTopic": "",
    "closeQos": "0",
    "closeRetain": "false",
    "closePayload": "",
    "closeMsg": {},
    "willTopic": "",
    "willQos": "0",
    "willRetain": "false",
    "willPayload": "",
    "willMsg": {},
    "userProps": "",
    "sessionExpiry": ""
  }
]

```